

Laboratório 01 - Características de Repositórios Populares

Lucas Flor Vilela
lucas.vilela@sga.pucminas.br

Marina Ferreira Sansão Cabalzar
marina.cabalzar@sga.pucminas.br

13 de março de 2026

[Repositório do projeto](#)

Resumo

O desenvolvimento de software de código aberto (open-source) tornou-se um pilar fundamental na engenharia de software moderna. Plataformas como o GitHub hospedam milhões de projetos, mas apenas uma pequena parcela atinge um alto nível de popularidade e engajamento. Este artigo apresenta um estudo empírico com o objetivo de caracterizar os 1.000 repositórios mais populares do GitHub. Para tanto, utilizou-se a API GraphQL do GitHub para extrair e analisar métricas fundamentais, incluindo a idade do repositório, volume de contribuições externas (Pull Requests), frequência de lançamentos (Releases) e atualizações, linguagens de programação predominantes e a taxa de resolução de problemas (Issues). A metodologia baseia-se na coleta automatizada de dados via script em Python e na sumarização quantitativa através de valores medianos. Espera-se que as análises forneçam insights valiosos sobre o nível de maturidade, as práticas de manutenção e o engajamento da comunidade por trás dos sistemas open-source de maior sucesso na atualidade.

1 Introdução e Motivação

Esta seção apresenta o contexto geral do estudo, bem como o problema de pesquisa que motivou a realização deste experimento, o porquê ele é necessário e algumas restrições e riscos.

1.1 Problema

O desenvolvimento de software de código aberto (open-source) revolucionou a maneira como sistemas são construídos, mantidos e distribuídos globalmente. Plataformas de hospedagem colaborativa, em especial o GitHub, concentram hoje dezenas de milhões de repositórios, servindo como a principal infraestrutura para a criação de bibliotecas, frameworks e sistemas complexos. No entanto, dentro desse vasto ecossistema, apenas uma parcela extremamente restrita de projetos atinge um alto nível de popularidade e adoção, métrica frequentemente representada pelo número de "estrelas" (*stars*) recebidas pela comunidade.

O **problema central** que motiva este estudo reside no desafio de compreender quais **fatores técnicos e colaborativos** distinguem esses sistemas de **sucesso absoluto** dos demais projetos. A **manutenção** de um repositório altamente popular **vai além da simples escrita de código**; ela configura um complexo desafio de engenharia de software e de **gestão** de produto. Sistemas de sucesso precisam orquestrar um volume massivo de **contribuições externas (Pull Requests)**, manter um ciclo ágil de *correções* e melhorias fechando chamados *Issues* de forma eficiente, e realizar **atualizações** e lançamentos (*Releases*) de forma contínua para manter a relevância tecnológica. Além disso, a **linguagem de programação** atua como um **fator determinante** para a **atração e retenção de mantenedores ativos**.

Dessa forma, a **motivação** é **investigar empírica e quantitativamente** as principais **características** que definem os **sistemas open-source mais populares** da atualidade. Entender os padrões de maturidade, frequência de atualização e engajamento comunitário desses 1.000 repositórios de topo fornece insights valiosos sobre as melhores práticas de manutenção e evolução colaborativa em projetos de software em larga escala.

1.2 Por que o Experimento é Necessário

A dependência global de infraestruturas de software baseadas em projetos *open-source* torna a análise desses ecossistemas uma necessidade crítica. Como empresas de todos os portes fundamentam seus produtos em ferramentas de código aberto, a estagnação ou o abandono de um repositório popular pode gerar um efeito cascata, comprometendo inúmeros sistemas dependentes. Contudo, o ecossistema do GitHub sofre com altas taxas de *abandonware* e esgotamento de mantenedores, exigindo que a engenharia de software substitua a intuição por dados empíricos na hora de avaliar a "saúde" e a sustentabilidade de um projeto.

Nesse contexto, este estudo se justifica ao utilizar técnicas de Mineração de Repositórios de Software para extrair um retrato quantitativo dos 1.000 repositórios de maior destaque no GitHub. Ao analisar métricas concretas — como tempo de vida, volume de contribuições e cadência de lançamentos —, o experimento fornece *insights* baseados em dados reais. Esses resultados são fundamentais tanto para guiar desenvolvedores na adoção de melhores práticas de governança, quanto para auxiliar empresas na mitigação de riscos ao incorporar soluções de terceiros em seus próprios produtos.

1.3 Restrições e Riscos

O desenvolvimento deste experimento apresenta algumas restrições que podem influenciar o escopo e os resultados obtidos. A primeira limitação está relacionada à fonte de dados utilizada. A coleta das informações foi realizada por meio da API GraphQL do GitHub, o que impõe restrições de taxa de requisições e pode limitar a quantidade de dados que podem ser extraídos em um determinado período de tempo. Além disso, algumas métricas disponíveis na plataforma podem não representar completamente o estado real de um projeto, uma vez que certas atividades da comunidade podem ocorrer fora do próprio repositório.

Outra restrição relevante refere-se à seleção da amostra analisada. O estudo considera apenas os 1.000 repositórios mais populares do GitHub, identificados principalmente pelo número de estrelas. Embora essa métrica seja amplamente utilizada para indicar popularidade, ela não necessariamente reflete a qualidade do software, sua sustentabilidade a longo prazo ou o impacto real do projeto no ecossistema de software.

Também existe a limitação relacionada às métricas analisadas. O experimento concentra-se em indicadores específicos, como idade do repositório, número de Pull Requests, frequência de Releases, linguagens predominantes e taxa de resolução de Issues. Apesar de essas métricas fornecerem uma visão quantitativa relevante, elas não capturam totalmente aspectos qualitativos importantes, como a qualidade do código, a organização da comunidade ou a complexidade do projeto.

Entre os principais riscos do estudo está a possibilidade de inconsistências ou mudanças nos dados do GitHub ao longo do tempo, já que repositórios podem receber novas contribuições, atualizações ou até mesmo serem removidos durante o período de coleta e análise. Além disso, scripts automatizados utilizados para extração dos dados podem enfrentar falhas técnicas ou limitações da própria API, o que pode impactar a completude da base analisada.

Por fim, há o risco de interpretação limitada dos resultados. Como o estudo baseia-se em uma análise quantitativa e em valores medianos das métricas coletadas, algumas particularidades de projetos específicos podem não ser totalmente capturadas. Ainda assim, as análises realizadas fornecem uma visão representativa das características predominantes entre os projetos open-source mais populares da plataforma.

2 Objetivos e Hipóteses

Nesta seção são apresentados os objetivos do experimento, as hipóteses formuladas e os critérios de sucesso para orientar a investigação proposta neste estudo.

2.1 Objetivo Principal

O objetivo principal deste experimento empírico é **investigar e caracterizar o perfil** de desenvolvimento e de manutenção dos **1.000 repositórios *open-source* mais populares** hospedados no **GitHub** (classificados pelo maior número de estrelas). Para alcançar esse propósito, o estudo visa

extrair e **analisar** quantitativamente um conjunto específico de **métricas** estruturais e colaborativas englobando a maturidade do projeto, o volume de colaborações externas, o ritmo de lançamentos (*releases*), a frequência de atualizações, as tecnologias predominantes e a eficiência na resolução de problemas (*issues*). Dessa forma, **busca-se traçar um panorama** baseado em dados empíricos sobre as práticas de engenharia que **sustentam** os sistemas de código aberto de maior **sucesso** na atualidade.

2.2 Hipóteses

Para orientar a análise dos dados coletados e responder às Questões de Pesquisa (RQs) propostas para este laboratório, as seguintes hipóteses informais foram estabelecidas *a priori*:

- **H01 (Maturidade - RQ 01):** Espera-se que os **sistemas mais populares sejam maduros** e possuam idade avançada. A premissa é que a construção de uma comunidade sólida e o acúmulo de milhares de estrelas no GitHub demandam tempo de consolidação.
- **H02 (Contribuição Externa - RQ 02):** Acredita-se que esses repositórios recebam um **volume massivo de contribuições externas**. Sistemas populares tendem a atrair desenvolvedores globais, refletindo-se em um **alto total de *Pull Requests*** aceitas.
- **H03 (Lançamentos - RQ 03):** Supõe-se que projetos de sucesso possuam um ciclo de integração ativo, lançando ***releases* com alta frequência** para disponibilizar novas funcionalidades e correções.
- **H04 (Atualizações - RQ 04):** Espera-se que o tempo até a última atualização seja muito curto, indicando **alta frequência de atualização** e evidenciando manutenção constante do projeto.
- **H05 (Linguagens - RQ 05):** A hipótese é que a **linguagem** primária dos repositórios reflita as **tendências do mercado de tecnologia** (como JavaScript, Python, TypeScript ou Java), facilitando a **atração de novos mantenedores**.
- **H06 (Resolução de Problemas - RQ 06):** Espera-se encontrar um **alto percentual de *issues* fechadas**, indicando governança eficiente e uma **comunidade ativa na resolução de bugs e suporte**.

2.3 Critérios de Sucesso

Por se tratar de um estudo empírico observacional baseado na mineração de repositórios, o êxito deste experimento não é medido pela alteração de uma métrica de produto, mas sim pela confiabilidade da execução metodológica e pela validade dos dados obtidos. Dessa forma, definem-se os seguintes critérios de sucesso para o laboratório:

- **Coleta e Integridade dos Dados:** Sucesso na automação da extração de dados via API GraphQL do GitHub, alcançando a amostra estipulada de 1.000 repositórios populares sem falhas críticas de conexão ou corrupção de dados estruturais.
- **Compleitude das Métricas:** Garantia de que as propriedades essenciais exigidas pelas Questões de Pesquisa (data de criação, data de atualização, linguagem primária, total de *releases*, *pull requests* aceitas e contagem de *issues*) foram devidamente recuperadas e estruturadas em formato tabular (.csv), permitindo o cálculo matemático preciso.
- **Resolução das Questões de Pesquisa:** Capacidade de calcular as medianas (para dados contínuos) [cite: 22] e as frequências (para dados categóricos) de forma a responder clara e objetivamente a todas as seis RQs propostas (RQ 01 a RQ 06), permitindo o confronto direto com as hipóteses estabelecidas.

3 Métricas

Para responder às questões de pesquisa propostas neste experimento, foram definidas métricas extraídas diretamente dos repositórios do GitHub. Essas métricas permitem avaliar características relacionadas à maturidade dos projetos, ao nível de colaboração da comunidade, à frequência de manutenção e às tecnologias utilizadas pelos sistemas open-source mais populares.

- A primeira métrica considerada é a **idade do repositório**, calculada a partir da **data de criação do projeto**. Essa métrica é utilizada para investigar se **sistemas populares tendem a ser mais antigos** e maduros ou se **projetos mais recentes também conseguem alcançar grande popularidade** dentro da plataforma.
- A segunda métrica analisada corresponde ao **total de pull requests aceitas**, utilizada para avaliar o **nível de contribuição externa**. O número de pull requests aceitas indica o **grau de participação da comunidade** no desenvolvimento do software e o **quanto os mantenedores aceitam** contribuições externas.
- A terceira métrica considerada é o **total de releases publicadas**. Essa métrica permite analisar se projetos populares mantêm um ciclo frequente de lançamentos, refletindo práticas de manutenção contínua e **evolução do software ao longo do tempo**.
- A quarta métrica analisada foi o **tempo desde a última atualização do repositório**, calculado a partir da **data do último commit**. Essa métrica permite identificar se os projetos mais populares **continuam sendo atualizados** com frequência ou se apresentam **sinais de estagnação**.
- A quinta métrica considerada foi a **linguagem** de programação primária. Essa métrica possibilita identificar **quais são mais utilizadas** entre os **projetos open-source mais populares**, permitindo observar tendências tecnológicas no GitHub.
- Por fim, a sexta métrica analisada foi a **taxa de resolução de issues**, calculada como a razão entre o número de issues fechadas e o total de issues registradas. Essa métrica permite avaliar a **eficiência na gestão de problemas reportados pela comunidade**, refletindo o nível de manutenção e suporte oferecido pelos mantenedores.

Em conjunto, essas métricas permitem construir uma visão quantitativa das características predominantes dos repositórios open-source mais populares do GitHub, fornecendo dados para análise das questões de pesquisa propostas neste estudo.

4 Ambientes e Materiais

Esta seção detalha as ferramentas, linguagens e especificações técnicas utilizadas para a extração, armazenamento e análise dos dados do experimento, garantindo a reprodutibilidade do estudo por outros pesquisadores.

4.1 Versões e Ferramentas

O experimento foi integralmente roteirizado e conduzido em um ambiente de desenvolvimento local utilizando a linguagem **Python** (versão 3.12+), escolhida por sua robustez e amplo suporte acadêmico na mineração de repositórios. O código-fonte foi desenvolvido e executado através da IDE **Visual Studio Code** (versão 1.86+).

Para a comunicação com a API e a modelagem matemática, o ambiente foi configurado com as seguintes bibliotecas e padrões de estruturação:

- **Bibliotecas de Coleta:** Utilizou-se a biblioteca nativa **json** para a manipulação dos retornos da API e a biblioteca **requests** (versão 2.31+) para orquestrar as requisições HTTP POST direcionadas ao *endpoint* GraphQL do GitHub.

- **Bibliotecas de Análise e Visualização:** Para a fase de sumarização, cálculo das medianas e geração de gráficos, empregaram-se as bibliotecas **pandas** (versão 2.2+) para a manipulação matricial dos dados e **matplotlib** (versão 3.8+) para a plotagem das figuras.
- **Armazenamento de Dados:** Para evitar o carregamento contínuo da API e manter o isolamento das etapas do experimento, os dados brutos extraídos foram persistidos localmente. Utilizou-se um arquivo de texto plano estruturado no formato **CSV** (*Comma-Separated Values*), utilizando o ponto e vírgula (;) como delimitador. A escolha do formato **.csv** assegura que a base de dados seja leve, portátil e facilmente consumida por ferramentas de análise estatística.

4.2 Dados de Entrada

Os dados que fundamentam este experimento foram extraídos diretamente da base oficial do **GitHub**, utilizando a **API GraphQL (v4)**. A escolha da arquitetura GraphQL, em detrimento da API REST tradicional (v3), justifica-se pela capacidade de realizar consultas precisas e aninhadas em uma única requisição HTTP, minimizando o tráfego de rede e evitando o problema de *over-fetching* (recebimento de dados desnecessários).

O conjunto de dados de entrada (*dataset*) foi rigorosamente definido pelos seguintes parâmetros de extração:

- **Amostra Populacional:** Os 1.000 repositórios públicos mais populares da plataforma, classificados em ordem decrescente pelo seu número de estrelas.
- **Filtro de Busca:** A *query* principal utilizou a string de busca `stars:>100 sort:stars-desc` para garantir que o *endpoint* retornasse apenas os repositórios de maior relevância mundial.
- **Propriedades Coletadas:** Para compor a base necessária para o cálculo das RQs, o *payload* JSON exigiu a devolução específica dos seguintes campos para cada repositório: nome do projeto, data de criação (`createdAt`), data da última atualização (`updatedAt`), linguagem primária (`primaryLanguage`), além das contagens totais (`totalCount`) das coleções de *Issues*, *Pull Requests* (exclusivamente aquelas com o status *MERGED*) e *Releases*.

Devido às restrições técnicas de proteção contra sobrecarga estabelecidas pelo GitHub (limite máximo de 100 nós de repositório por requisição), o montante total de dados de entrada não pôde ser resgatado de uma só vez. Dessa forma, a entrada do experimento dependeu de um sistema de paginação dinâmica, utilizando cursores de navegação (`endCursor` e `hasNextPage`) para encadear os lotes de dados de 10 em 10 páginas até totalizar os 1.000 projetos.

5 Procedimento (passo a passo)

Nesta seção, é detalhado o roteiro de execução do experimento, permitindo sua completa reprodução por avaliadores ou terceiros, além de apresentar o fluxo de trabalho operacional adotado ao longo do projeto.

5.1 Como Rodar o Experimento

Para garantir a execução correta e a replicação dos resultados obtidos, o pesquisador deve seguir os passos descritos abaixo:

1. **Configuração do Ambiente:** Certifique-se de possuir o Python instalado na máquina. Em seguida, instale as bibliotecas necessárias executando o comando no terminal: `pip install requests pandas matplotlib`.
2. **Autenticação na API:** Gere um *Personal Access Token* na sua conta do GitHub. Insira este *token* no cabeçalho de autorização (*headers*) do script de coleta para permitir as requisições à API GraphQL.
3. **Extração de Dados:** Execute o script responsável pela comunicação com a API. O código fará as requisições paginadas, extrairá os nós JSON e criará automaticamente o arquivo `repositorios.csv` contendo a amostra bruta na raiz do diretório.

4. **Análise e Visualização:** Com a base de dados estruturada, execute o script de análise (`visualizacao.py`). O terminal exibirá o cálculo das medianas exigidas pelas Questões de Pesquisa (RQs) e o código salvará localmente as imagens dos gráficos (`.png`) gerados.

5.2 Checklist Operacional (Fluxo de Sprints)

A garantia da integridade dos dados e o controle operacional do experimento foram gerenciados através de um ciclo iterativo, dividido estrategicamente nas seguintes *Sprints*:

- **Sprint 1 (Planejamento e Preparação):**
 - Definição formal das Questões de Pesquisa e das métricas-alvo.
 - Leitura da documentação oficial da API GraphQL (v4) do GitHub.
 - Construção e teste da *query* de busca utilizando o *GitHub GraphQL API Explorer*, assegurando que filtros críticos (como trazer apenas PRs com status *MERGED*) funcionassem corretamente antes de ir para o código.
- **Sprint 2 (Coleta e Integridade):**
 - Implementação do script de coleta em Python.
 - Lógica de paginação implementada (controle via `endCursor` e `hasNextPage`) para contornar a restrição de 100 resultados por requisição imposta pelo GitHub.
 - Validação de integridade do arquivo gerado: auditoria no arquivo `repositorios.csv` para garantir a presença exata de 1.000 registros únicos, sem dados corrompidos ou colunas vazias.
- **Sprint 3 (Análise e Documentação):**
 - Escrita do script de visualização com o `pandas` para higienizar os formatos de data e calcular as fórmulas derivadas (idade em anos, dias sem atualização e razão de issues fechadas).
 - Geração e exportação dos gráficos de forma isolada com o `matplotlib`.
 - Execução da análise cruzada adicional (Bônus da RQ 07) integrando os dados com o relatório *GitHub Octoverse*.
 - Consolidação, revisão e formatação do artigo científico na plataforma Overleaf.

6 Resultados

Nesta seção, são apresentados os dados extraídos dos 1.000 repositórios mais populares do GitHub. A análise quantitativa baseou-se no cálculo de valores medianos para contornar o efeito de *outliers* (repositórios com valores extremos) e na extração de frequências para dados categóricos.

6.1 Tabelas e Gráficos Principais

Para facilitar a compreensão da distribuição dos dados, a sumarização visual do experimento foi dividida entre o perfil de idade dos projetos, o engajamento da comunidade e as tecnologias adotadas.

6.2 Resultados por Segmento (Questões de Pesquisa)

A extração das métricas primárias permitiu responder diretamente às Questões de Pesquisa (RQs) propostas. Os resultados consolidados da amostra revelam o seguinte panorama:

- **RQ 01 (Idade):** Os repositórios analisados apresentam uma maturidade consolidada, com uma mediana de idade de **8,40 anos**. A distribuição completa dessa faixa etária pode ser observada na Figura 1.
- **RQ 02 (Contribuição Externa):** O engajamento de desenvolvedores externos é massivo. A amostra registrou uma mediana de **739 Pull Requests (PRs)** aceitos e mesclados (*merged*) à base de código principal.

- **RQ 03 (Lançamentos):** A cadência de entrega de valor mostrou-se consistente, com os projetos apresentando uma mediana de **40 *releases*** publicadas ao longo de sua existência.
- **RQ 04 (Atualização):** O ecossistema demonstrou ser altamente ativo, registrando uma mediana de apenas **7 dias** decorridos desde a última atualização no repositório, padrão detalhado na Figura 2.
- **RQ 05 (Linguagem Primária):** Houve uma clara dominância de linguagens voltadas para web e dados. Conforme ilustrado na Figura 3, o **Python** liderou isoladamente (presente em 200 repositórios), seguido por **TypeScript** (160) e **JavaScript** (115). Outras linguagens de destaque incluíram Go (77) e Rust (54).
- **RQ 06 (Resolução de Problemas):** A eficiência da governança mostrou-se elevada, com os mantenedores conseguindo fechar, em mediana, **86,75%** de todas as *issues* abertas pela comunidade, cuja concentração é evidenciada na Figura 4.

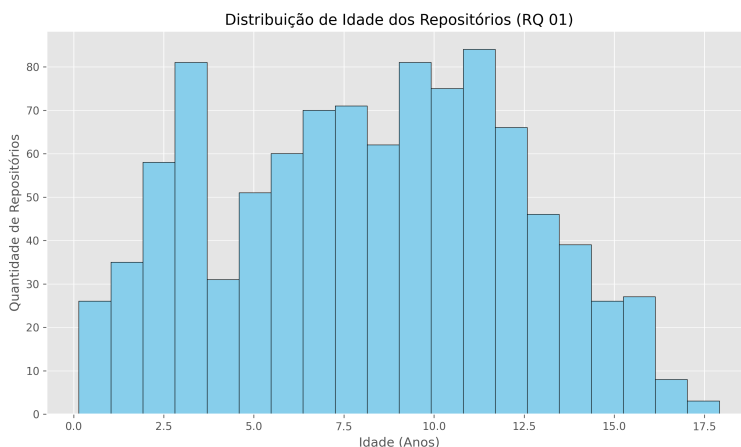


Figura 1: Distribuição de Idade dos Repositórios (RQ 01).

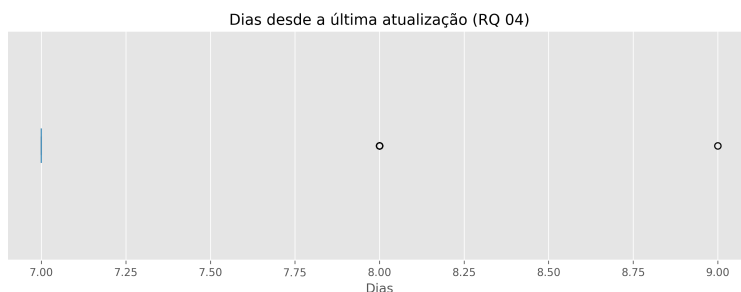


Figura 2: Dias decorridos desde a última atualização (RQ 04).

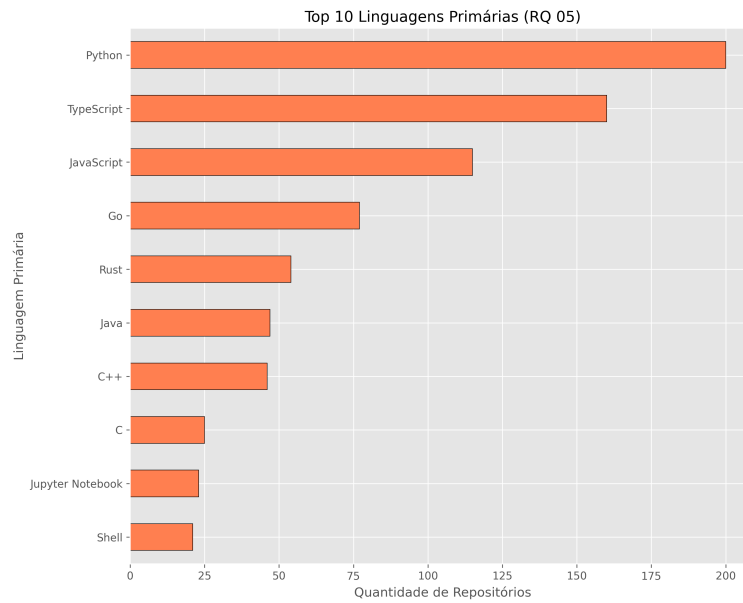


Figura 3: Top 10 Linguagens Primárias mais utilizadas na amostra (RQ 05).

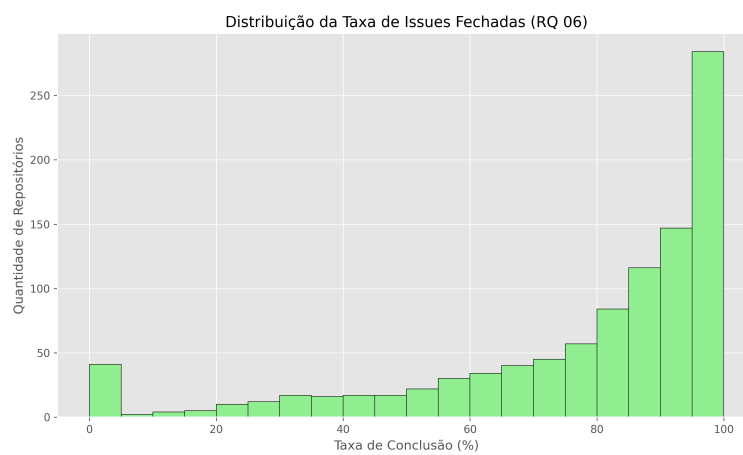


Figura 4: Distribuição da Taxa de Issues Fechadas (RQ 06).

6.3 Observações Operacionais e Análises Adicionais (RQ 07)

Como desdobramento do estudo, foi realizada uma segmentação extra (RQ 07) baseada no relatório *GitHub Octoverse* [1], que aponta uma mudança estrutural no desenvolvimento de software impulsionada pela Inteligência Artificial. A reportagem destaca a ascensão meteórica do **TypeScript** (que ganhou protagonismo devido à segurança de tipagem exigida por IAs gerativas) e a contínua dominância do **Python** e **JavaScript**, além de linguagens consolidadas como **Java**, **C++** e **C#**.

O objetivo desta etapa foi isolar os repositórios escritos especificamente neste grupo de linguagens de ponta apontadas pelo GitHub e compará-los com o restante da amostra.

A análise comparativa revelou que os sistemas escritos nas linguagens citadas pela reportagem apresentaram uma mediana de **938** PRs aceitos, contra **494** PRs do grupo de outras linguagens. Em relação à cadência, o grupo líder publicou uma mediana de **62** *releases* e registrou **7** dias sem atualização, enquanto o segundo grupo marcou apenas **16** *releases* e os mesmos **7** dias desde a última atualização.

Estes resultados corroboram fortemente a tese do *Octoverse*: a adoção de tecnologias ligadas ao ecossistema de IA e desenvolvimento web moderno não reflete apenas uma preferência de sintaxe, mas impacta de forma direta na atração de uma comunidade massiva de contribuidores externos e em uma cadência de entrega (*releases*) substancialmente maior.

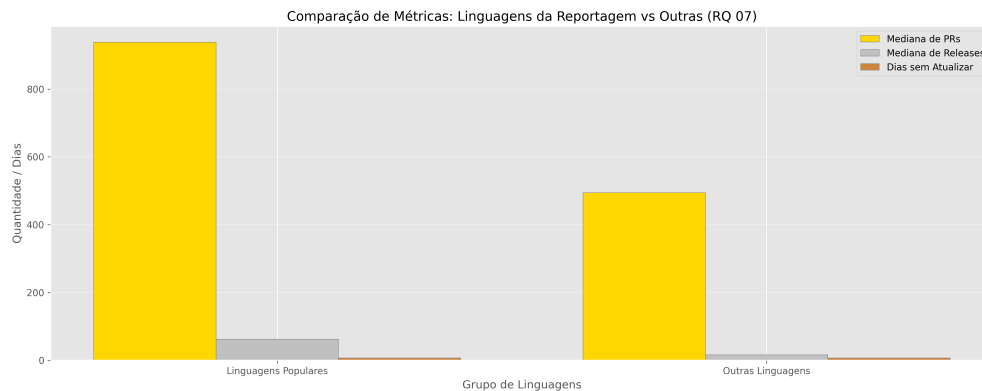


Figura 5: Comparação de Métricas: Linguagens do Octoverse vs Outras (RQ 07).

Referências

- [1] GitHub. Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1, 2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>. Acesso em: 13 mar. 2026.